

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

Lab Organización y Arquitectura de Computadoras
Grupo: 03

Práctica 4

P R E S E N T A
Rea Alberto Cristian | 318273130
Ortega de la Paz Rafael | 420054085

PROFESOR DE LABORATORIO
Ricardo Daniel Vega Gutierrez



Fecha de entrega: 25 de Octubre de 2026
Semestre 2027-1

Objetivo

Diseñar una máquina de estados de ocho estados, cuatro entradas y cuatro salidas a partir de su carta ASM, obtener el contenido de la memoria con el método de direccionamiento entrada-estado, implementarla en Quartus y comprobar su funcionamiento en la simulación y en la tarjeta DE10-Lite.

Introducción

En la Práctica 3 la máquina de estados se armó con flip-flops tipo D y la lógica del estado siguiente salió de mapas de Karnaugh. En esta práctica el camino es otro: toda la información de la máquina (a qué estado se pasa y qué salidas se activan) se guarda en una memoria, y el circuito solo se encarga de leerla en cada ciclo de reloj.

El método que usamos es el direccionamiento entrada-estado (Savage Carmona, 2017). La dirección de la memoria es únicamente el estado presente, y cada localidad guarda cinco campos: la parte de prueba, que dice qué entrada se revisa en ese estado; la liga falsa y la liga verdadera, que son los estados siguientes cuando la entrada vale 0 o 1; y dos campos de salidas, uno para cuando la entrada vale 0 y otro para cuando vale 1. Por eso solo se puede probar una entrada por estado, y los estados que no dependen de ninguna entrada ponen la misma liga en las dos ramas.

La carta ASM no representa un sistema físico en particular, se diseñó para cumplir lo que pide la práctica: ocho estados, cuatro entradas, cuatro salidas (dos con lógica estándar y dos con lógica negada) y tres salidas condicionales. A partir de ella calculamos el tamaño de la memoria, llenamos la tabla con su contenido y sacamos tres mapas de Karnaugh de esa tabla para revisar la liga falsa.

Desarrollo

Carta ASM

La carta tiene ocho estados, E0 a E7, y cuatro entradas, X, Y, Z y W. S_1 y S_4 usan lógica estándar, así que se activan con 1, y $\bar{S}_2$ y $\bar{S}_3$ usan lógica negada, es decir, se activan con 0. Hay tres salidas condicionales: S_1 cuando Y = 0, $\bar{S}_2$ cuando Y = 1 y S_1 cuando Z = 1. La Figura 1 muestra la carta pasada a limpio y la Tabla 3 resume lo que hace cada estado.

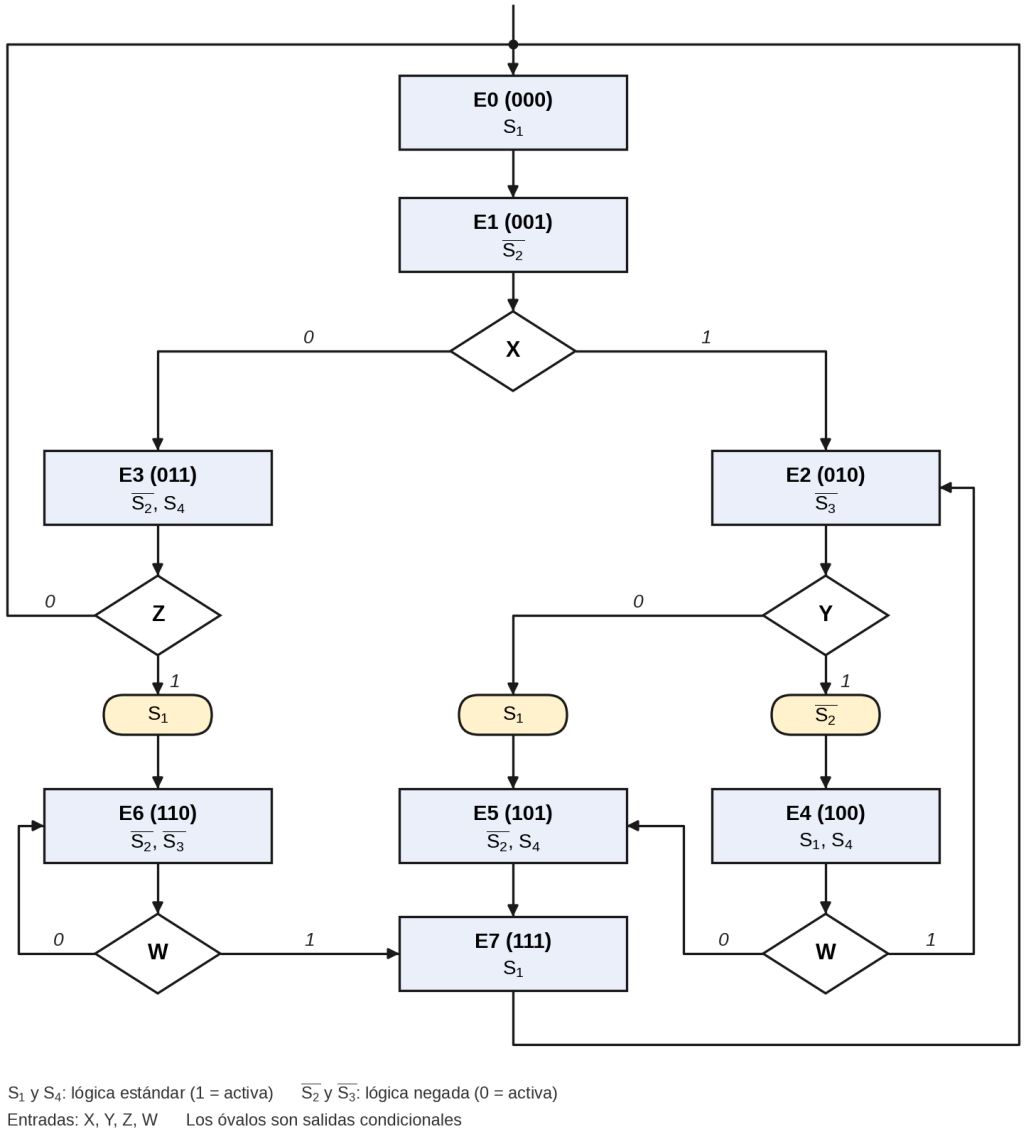


Figura 1. Carta ASM de la máquina (S_1 y S_4 activas en 1; $\bar{S}_2$ y $\bar{S}_3$ activas en 0).

Antes de armar la tabla asignamos un código binario a cada entrada, a cada estado y a las salidas. Los estados E0, E5 y E7 no dependen de ninguna entrada, así que en ellos se prueba la entrada auxiliar Qx; como sus dos ligas apuntan al mismo estado, el valor de Qx no cambia nada. A los estados se les dio código binario consecutivo, de E0 = 000 a E7 = 111.

Entrada	Código de prueba
X	000
Y	001
Z	010
W	011
Qx (auxiliar)	100

Tabla 1. Asignación binaria de las entradas.

En las salidas se guarda el nivel real de cada pin. Una salida que no se usa en un estado queda con $S_1 = 0$, $\bar{S}_2 = 1$, $\bar{S}_3 = 1$ y $S_4 = 0$ (el patrón 0110); cuando se activa, S_1 y S_4 pasan a 1 y $\bar{S}_2$ y $\bar{S}_3$ pasan a 0 (Tabla 2).

Salida	Tipo	Nivel inactiva	Nivel activa
S_1	Estándar	0	1
$\bar{S}_2$	Negada	1	0
$\bar{S}_3$	Negada	1	0
S_4	Estándar	0	1

Tabla 2. Convención de las salidas.

Estado	Código	Salidas del estado	Prueba	Siguiente estado
E0	000	S_1	Qx (sin prueba)	E1
E1	001	$\bar{S}_2$	X	X = 0 → E3; X = 1 → E2
E2	010	$\bar{S}_3$	Y	Y = 0 (S_1) → E5; Y = 1 ($\bar{S}_2$) → E4
E3	011	$\bar{S}_2$, S_4	Z	Z = 0 → E0; Z = 1 (S_1) → E6
E4	100	S_1, S_4	W	W = 0 → E5; W = 1 → E2
E5	101	$\bar{S}_2$, S_4	Qx (sin prueba)	E7
E6	110	$\bar{S}_2$, $\bar{S}_3$	W	W = 0 → E6; W = 1 → E7
E7	111	S_1	Qx (sin prueba)	E0

Tabla 3. Estados, código, salidas y trayectorias de la carta ASM.

Tamaño de la memoria

La dirección de la memoria es solo el estado presente, y con 8 estados hacen falta n = 3 bits. Cada localidad guarda la parte de prueba (3 bits, para codificar las cuatro entradas y Qx), la liga falsa (3 bits), la liga verdadera (3 bits), las salidas falsas (4 bits) y las salidas verdaderas (4 bits), así que m = 3 + 3 + 3 + 4 + 4 = 17 bits.

$$\textbf{Memoria} = 2^n \times m = 2^3 \times 17 = 8 \times 17 = 136 \text{ bits}$$

Son 8 localidades de 17 bits. Los 8 códigos que se pueden formar con 3 bits ya los usa un estado de la carta, entonces no quedaron combinaciones sin usar que hubiera que llenar.

Tabla de transiciones y contenido de memoria

La Tabla 4 es la tabla de transiciones. Las salidas van en el orden S_1 $\bar{S}_2$ $\bar{S}_3$ S_4. Las salidas falsas son las que se activan cuando la entrada probada vale 0 y las verdaderas cuando vale 1. En casi todos los estados son iguales; solo cambian en E2 y en E3, que son los que tienen salidas condicionales. Por ejemplo, E2 activa $\bar{S}_3$ y, si Y = 0, además S_1, por eso sus salidas falsas son 1 1 0 0; si Y = 1 activa $\bar{S}_3$ y $\bar{S}_2$ y quedan 0 0 0 0 en las verdaderas.

Estado actual	Prueba	Liga falsa	Liga verdadera	Salidas falsas				Salidas verdaderas			
				S_1	$\bar{S}_2$	$\bar{S}_3$	S_4	S_1	$\bar{S}_2$	$\bar{S}_3$	S_4
000 (E0)	100	001	001	1	1	1	0	1	1	1	0
001 (E1)	000	011	010	0	0	1	0	0	0	1	0
010 (E2)	001	101	100	1	1	0	0	0	0	0	0
011 (E3)	010	000	110	0	0	1	1	1	0	1	1
100 (E4)	011	101	010	1	1	1	1	1	1	1	1
101 (E5)	100	111	111	0	0	1	1	0	0	1	1
110 (E6)	011	110	111	0	0	0	0	0	0	0	0
111 (E7)	100	000	000	1	1	1	0	1	1	1	0

Tabla 4. Tabla de transiciones (contenido de la memoria por campos).

Si se pegan los cinco campos de cada renglón queda la palabra de 17 bits de cada localidad (Tabla 5). Ese es el contenido que se escribe en la memoria.

Dirección	Estado	Palabra de 17 bits
000	E0	10000100111101110
001	E1	00001101000100010
010	E2	00110110011000000
011	E3	01000011000111011
100	E4	01110101011111111
101	E5	10011111100110011
110	E6	01111011100000000
111	E7	10000000011101110

Tabla 5. Contenido de la memoria (prueba, liga falsa, liga verdadera, salidas falsas y salidas verdaderas).

El mismo contenido en el formato del archivo .mif con el que Quartus inicializa la memoria:

```
DEPTH = 8;
WIDTH = 17;
ADDRESS_RADIX = BIN;
DATA_RADIX = BIN;
CONTENT BEGIN
  000 : 10000100111101110;
  001 : 00001101000100010;
  010 : 00110110011000000;
  011 : 01000011000111011;
  100 : 01110101011111111;
  101 : 10011111100110011;
  110 : 01111011100000000;
  111 : 10000000011101110;
END;
```

Mapas de Karnaugh

De las columnas de la Tabla 4 sacamos tres mapas, los de los tres bits de la liga falsa (LF2, LF1 y LF0). Las variables son el estado actual A B C, donde A es el bit más significativo, y las columnas van en código Gray para que las celdas vecinas cambien un solo bit. En las ecuaciones la barra sobre una letra indica negación y el punto es AND.

Mapa de LF2

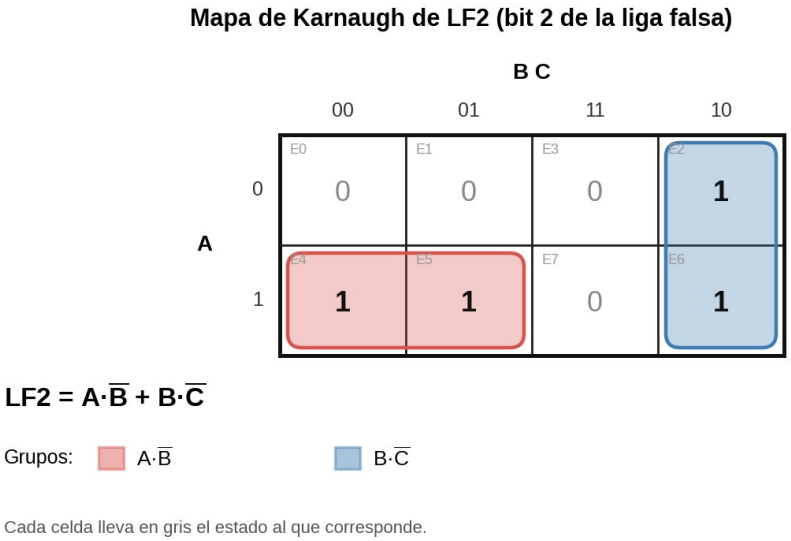


Figura 2. Mapa de Karnaugh de LF2 con sus grupos y su ecuación.

LF2 vale 1 en E2, E4, E5 y E6. Los estados E4 y E5 forman un grupo (A·B̄) y E2 con E6 forman otro (B·C̄). Los demás estados valen 0, así que no hace falta ningún otro término.

Mapa de LF1

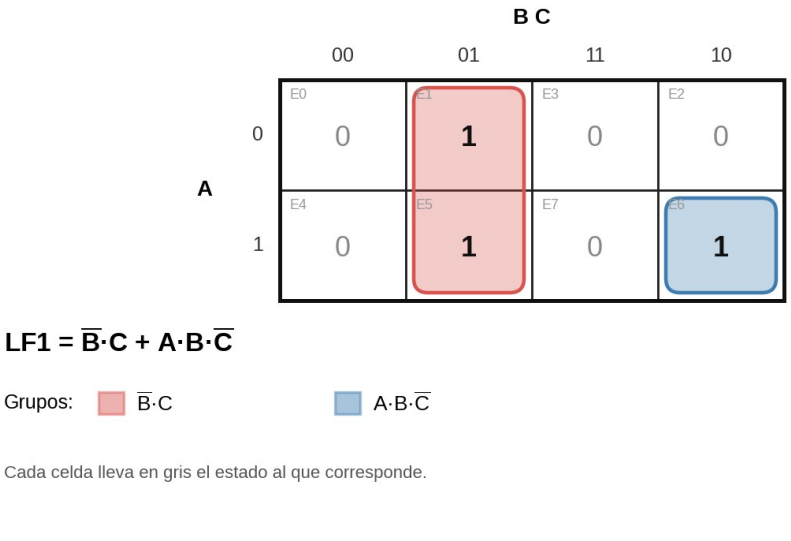


Figura 3. Mapa de Karnaugh de LF1 con sus grupos y su ecuación.

LF1 vale 1 en E1, E5 y E6. E1 y E5 se agrupan en $\bar{B}\cdot C$. El 1 de E6 no tiene vecinos con 1, por eso queda como un término completo, $A\cdot B\cdot \bar{C}$.

Mapa de LF0

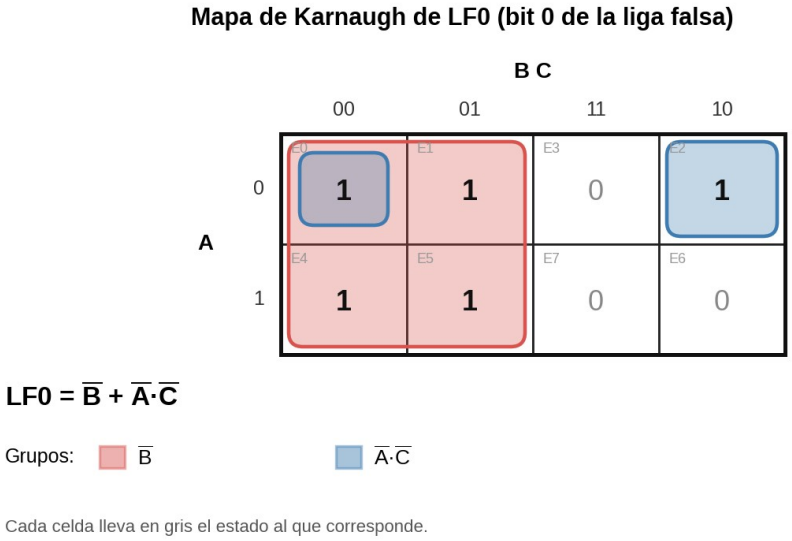


Figura 4. Mapa de Karnaugh de LF0 con sus grupos y su ecuación.

LF0 vale 1 en E0, E1, E2, E4 y E5. Los estados E0, E1, E4 y E5 forman un grupo de cuatro ($\bar{B}$). El estado E2 se agrupa con E0 porque la columna 10 es vecina de la 00 ($\bar{A}\cdot\bar{C}$), por eso ese grupo aparece partido en el mapa.

Evaluamos las tres ecuaciones en los 8 estados y dan los mismos valores que la columna de liga falsa de la Tabla 4.

Implementación en Quartus

El direccionamiento entrada-estado se armó en Quartus con los bloques del diagrama de direccionamiento entrada-estado modificado que viene en el manual de la práctica. Una memoria ROM de 8 palabras de 17 bits, inicializada con el archivo .mif de arriba, recibe como dirección la salida del registro de estado. De la palabra se separan los cinco campos: la prueba entra al selector de entradas, las dos ligas al selector de liga y los dos campos de salidas al selector de salidas.

El selector de entradas es un multiplexor de 8 a 1 controlado por el campo de prueba: con 000, 001, 010 y 011 deja pasar X, Y, Z y W, y con 100 deja pasar la entrada auxiliar Qx, que se dejó fija en 1. Las combinaciones 101, 110 y 111 de la prueba no se usan. La salida de este multiplexor controla al selector de liga y al selector de salidas: si vale 0 pasan la liga falsa y las salidas falsas, y si vale 1 pasan la liga verdadera y las salidas verdaderas. La liga elegida se guarda en un registro de tres flip-flops D, que es la dirección de la siguiente lectura, y las salidas elegidas se guardan en otro registro de cuatro bits. Los dos registros comparten el mismo reloj.

Igual que en la Práctica 3, el reloj es el botón KEY0 para poder ver cada transición, y el reset es KEY1, que deja la máquina en E0. Las entradas X, Y, Z y W se conectaron a los interruptores SW0 a SW3, S_1 y S_4 a LEDR0 y LEDR1, y $\bar{S}_2$ y $\bar{S}_3$ a los segmentos a y d del HEX0, que encienden con 0, de modo que cada segmento prende cuando la salida está activa. Los pines están en la Tabla 6 (Terasic, s. f.).

Puerto	Pin	Tipo	Uso
KEY[0]	PIN_B8	Entrada	Reloj manual (flanco de bajada al presionar)
KEY[1]	PIN_A7	Entrada	Reset, activo en bajo
SW[0]	PIN_C10	Entrada	X
SW[1]	PIN_C11	Entrada	Y
SW[2]	PIN_D12	Entrada	Z
SW[3]	PIN_C12	Entrada	W
LEDR[0]	PIN_A8	Salida	S_1
LEDR[1]	PIN_A9	Salida	S_4
HEX0[0]	PIN_C14	Salida	Segmento a: $\bar{S}_2$ (negada)
HEX0[3]	PIN_C16	Salida	Segmento d: $\bar{S}_3$ (negada)

Tabla 6. Asignación de pines.

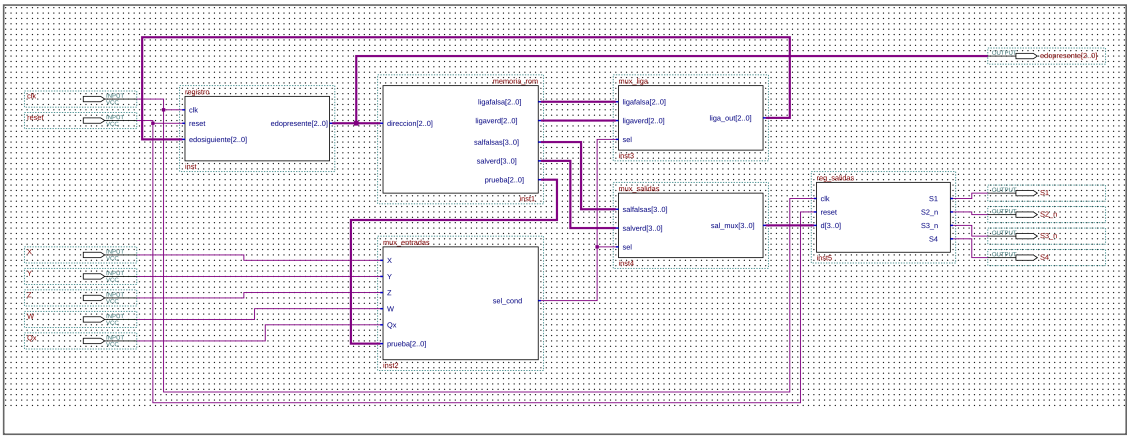


Figura 5. Diagrama del direccionamiento entrada-estado en Quartus.

Compilación, simulación y pruebas en la tarjeta

Compilación. Se creó el proyecto en Quartus para el dispositivo MAX 10 10M50DAF484C7G de la DE10-Lite y se compiló con Start Compilation.

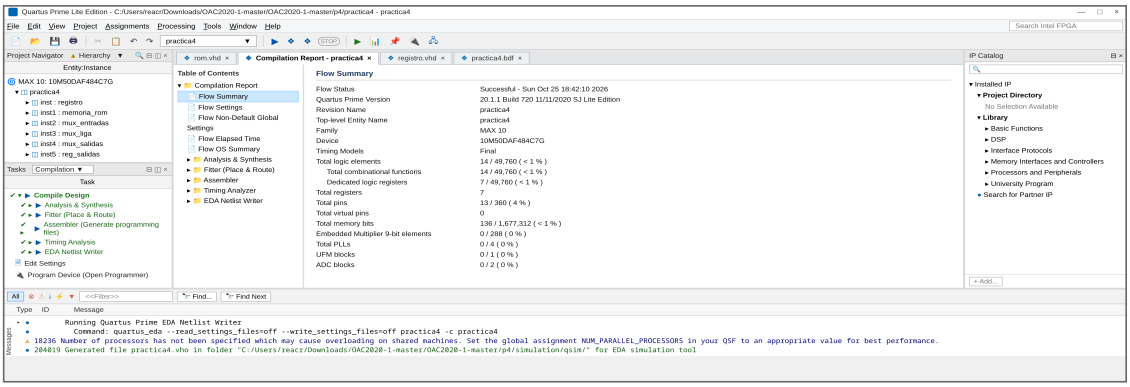


Figura 6. Resultado de la compilación en Quartus.

Simulación. En la simulación deben aparecer el contenido de la memoria y el estado presente, además de las entradas y las salidas. Se aplica primero el reset y después la secuencia de la Tabla 7, con un pulso de reloj por paso. La secuencia pasa por cada una de las 13 flechas de la carta ASM al menos una vez, y las salidas de cada paso son las que corresponden al estado actual y al valor de la entrada probada.

Paso	Entradas				Estado		Salidas				Transición que se prueba
	X	Y	Z	W	Actual	Siguiente	S_1	$\bar{S}_2$	$\bar{S}_3$	S_4	
1	0	0	0	0	E0	E1	1	1	1	0	E0 → E1
2	0	0	0	0	E1	E3	0	0	1	0	E1 → E3 (X = 0)
3	0	0	0	0	E3	E0	0	0	1	1	E3 → E0 (Z = 0)
4	0	0	0	0	E0	E1	1	1	1	0	E0 → E1
5	1	0	0	0	E1	E2	0	0	1	0	E1 → E2 (X = 1)
6	0	1	0	0	E2	E4	0	0	0	0	E2 → E4 (Y = 1), sale $\bar{S}_2$
7	0	0	0	1	E4	E2	1	1	1	1	E4 → E2 (W = 1)
8	0	0	0	0	E2	E5	1	1	0	0	E2 → E5 (Y = 0), sale S_1
9	0	0	0	0	E5	E7	0	0	1	1	E5 → E7
10	0	0	0	0	E7	E0	1	1	1	0	E7 → E0
11	0	0	0	0	E0	E1	1	1	1	0	E0 → E1
12	0	0	0	0	E1	E3	0	0	1	0	E1 → E3 (X = 0)
13	0	0	1	0	E3	E6	1	0	1	1	E3 → E6 (Z = 1), sale S_1
14	0	0	0	0	E6	E6	0	0	0	0	E6 se queda (W = 0)
15	0	0	0	1	E6	E7	0	0	0	0	E6 → E7 (W = 1)
16	0	0	0	0	E7	E0	1	1	1	0	E7 → E0
17	0	0	0	0	E0	E1	1	1	1	0	E0 → E1
18	1	0	0	0	E1	E2	0	0	1	0	E1 → E2 (X = 1)
19	0	1	0	0	E2	E4	0	0	0	0	E2 → E4 (Y = 1), sale $\bar{S}_2$
20	0	0	0	0	E4	E5	1	1	1	1	E4 → E5 (W = 0)

Tabla 7. Secuencia de prueba y resultado esperado.

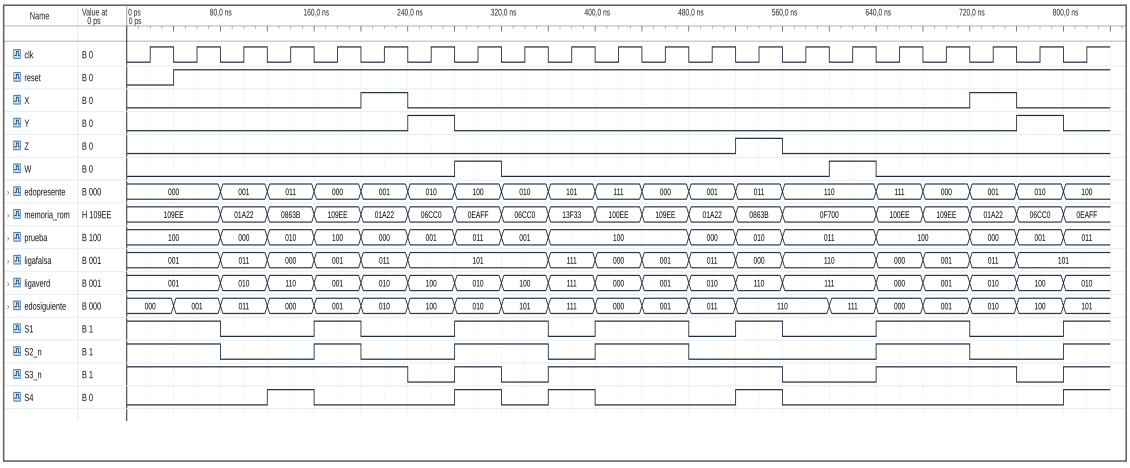


Figura 7. Simulación de la secuencia de prueba.

Pruebas en la tarjeta. Con la compilación terminada se abre el Programmer, se selecciona el cable USB-Blaster en modo JTAG y se carga el archivo .sof en el 10M50DAF484. Con la tarjeta programada se repite la secuencia de la Tabla 7: reset con KEY1, entradas con SW0 a SW3 y un pulso de KEY0 por cada paso.

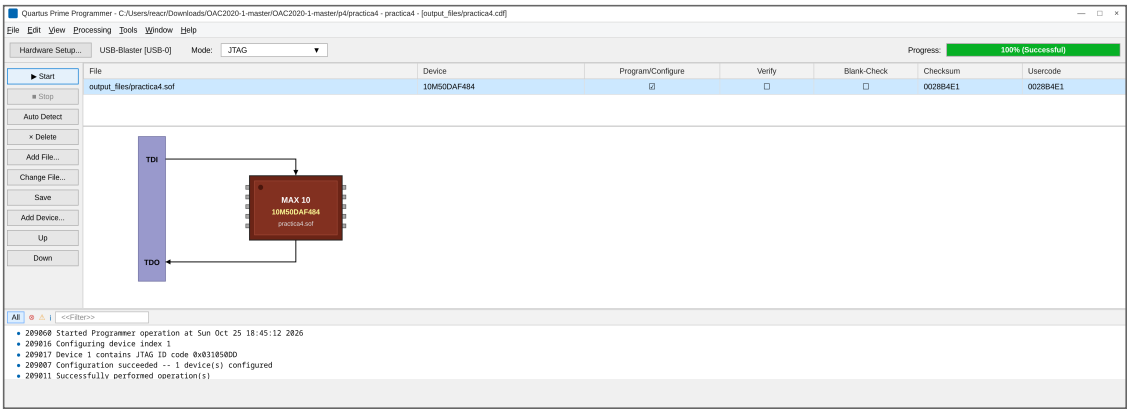


Figura 8. Programación y prueba de la DE10-Lite.

Conclusiones

Rea Alberto Cristian. Con esta práctica entendí mejor cómo una máquina de estados se puede guardar completa en una memoria. Lo que más cuidado me pidió fue llenar la tabla, porque cada renglón junta cinco campos y un bit mal puesto cambia el estado siguiente o una salida. Las salidas con lógica negada fueron lo más fácil de equivocar: $\bar{S}_2$ y $\bar{S}_3$ valen 0 cuando están activas y al principio uno tiende a poner 1. En E2 y E3 también hubo que separar las salidas falsas de las verdaderas, porque ahí lo que sale depende de la entrada. Los mapas de Karnaugh de la liga falsa me sirvieron para revisar la tabla: las tres ecuaciones coincidieron con la columna en los ocho estados. Comparado con la Práctica 3, aquí no hay que simplificar la lógica de cada flip-flop, pero sí hay que ser muy ordenado con el contenido de la memoria.

Ortega de la Paz Rafael. Lo que más me interesó fue ver cómo cambia el tamaño de la memoria según el método. Aquí la dirección es solo el estado presente (3 bits), entonces son 8 localidades aunque haya cuatro entradas; el costo es que cada palabra mide 17 bits y que en cada estado solo se puede probar una entrada. Por eso E0, E5 y E7, que no dependen de ninguna entrada, usan la entrada auxiliar Qx con las dos ligas iguales. Con la tabla de contenido ya hecha, armar el circuito en Quartus se reduce a escribir el archivo .mif y conectar los selectores, cosa que me pareció más directa que escribir las ecuaciones de cada flip-flop como en la práctica anterior. También me quedo con que conviene tener el estado presente a la vista en la simulación, porque ahí se ve de inmediato si la palabra que se leyó es la que esperábamos.

Referencias

Savage Carmona, J. (2017). Diseño de microprocesadores. Recuperado de http://profesores.fi-b.unam.mx/luist/archivos/MicroprocesadoresSavage.pdf

Terasic. (s. f.). DE10-Lite user manual. Terasic Technologies.